

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	Assessment:	Experiments
Weightage:	45%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	M.KOUSHIK KUMAR REDDY	Reg. No.:	192372185
Section / Batch:	CSE AI	Date:	11/8/2026

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Construct a pipeline scenario containing dependent instructions, branch instructions, and competing resource requirements, and apply techniques such as pipeline stalling, operand forwarding, branch prediction, and resource duplication to minimize performance losses.
2. Develop a computer system model using a hierarchical bus structure to transfer data among the CPU, memory, and I/O devices, and analyse how bus priority, bandwidth, and transfer conflicts affect overall system performance.
3. Develop a processor model that supports multiple instruction issue, out-of-order execution, and speculative execution, and analyse how instruction scheduling, execution units, and simultaneous multithreading affect instruction throughput and processor resource utilization.
4. Design and simulate a processor data path that performs register-to-register data transfers and arithmetic operations such as increment, decrement, addition, and subtraction, and analyse the control signals and execution sequence involved in each operation.
5. Simulate a pipelined processor executing a sequence of arithmetic, memory, and branch instructions, and calculate the pipeline execution time, throughput, and speedup by comparing the results with sequential instruction execution.

Code of Conduct

I M.KOUSHIK REDDY /192372185 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Course Coordinator Faculty Incharge:-

Processor Organization / Computer Architecture

Experiment 1: Pipeline Hazard Mitigation

Aim

To construct a pipeline scenario containing dependent instructions, branch instructions, and competing resource requirements, and apply pipeline stalling, operand forwarding, branch prediction, and resource duplication to minimize performance losses.

Requirements

- Computer
- CPU / processor simulator or diagram tool
- 5-stage pipeline model
- Instructions: ADD, SUB, BEQ, LW, SW
- Pipeline stages: IF, ID, EX, MEM, WB

Procedure

Step 1: Define the Pipeline

Use a standard 5-stage pipeline:

IF → ID → EX → MEM → WB

Where: IF – Instruction Fetch, ID – Instruction Decode, EX – Execute, MEM – Memory Access, WB – Write Back

Step 2: Enter Instructions

Use this instruction sequence:

```
ADD R1, R2, R3
SUB R4, R1, R5
BEQ R4, R6, LABEL
LW R7, 0(R8)
SW R9, 0(R8)
```

Step 3: Identify Hazards

Data Hazard:

```
ADD R1, R2, R3
SUB R4, R1, R5
```

ADD produces R1, while SUB immediately requires R1.

Step 4: Apply Operand Forwarding

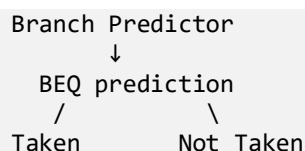
Instead of waiting for ADD to write R1 into the register file:

ADD → ALU Result → Forwarding Unit → SUB ALU

This eliminates unnecessary stalls.

Step 5: Handle Branch Hazard

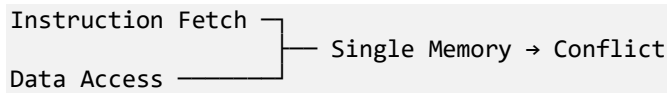
BEQ depends on R4 produced by SUB. Use a 2-bit dynamic branch predictor:



Correct prediction → continue execution. Incorrect prediction → flush incorrectly fetched instructions.

Step 6: Handle Structural Hazard

If one memory is used for both instructions and data:



Use separate Instruction Memory and Data Memory. This is resource duplication.

Step 7: Apply Stalling

A stall is required when forwarding cannot solve the dependency. Example:

```
LW R1, 0(R2)
SUB R4, R1, R5
```

The loaded value may not be available soon enough, so insert:

```
LW → SUB → BUBBLE
```

Result

The pipeline hazards are minimized using the following techniques:

Hazard	Technique
Data hazard	Operand forwarding
Load-use hazard	Pipeline stall
Control hazard	Branch prediction
Structural hazard	Resource duplication

Experiment 2: Hierarchical Bus Structure

Aim

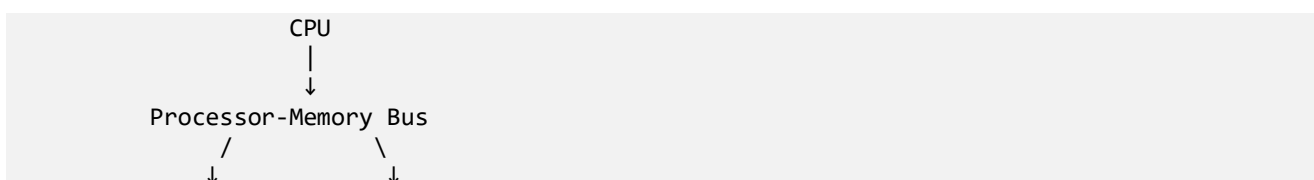
To develop a computer system model using a hierarchical bus structure for transferring data among CPU, memory, and I/O devices, and to analyze bus priority, bandwidth, and transfer conflicts.

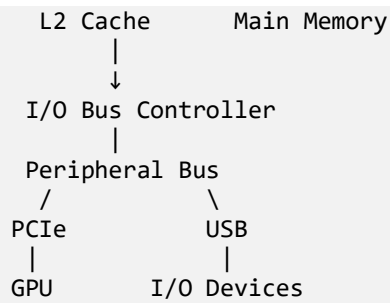
Requirements

- CPU model
- Main memory
- Cache
- I/O devices
- Bus controller
- Bus arbiter
- System bus
- I/O bus

Procedure

Step 1: Create the System





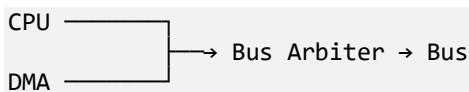
Step 2: Define Bus Hierarchy

High-speed bus: CPU ↔ Cache ↔ Main Memory

Lower-speed I/O bus: I/O Controller ↔ PCIe / USB ↔ Devices

Step 3: Add Bus Arbitration

Suppose CPU and DMA request the bus simultaneously:



The arbiter decides which device gets access.

Step 4: Select Priority

Fixed priority: CPU > DMA > I/O

Round-robin: CPU → DMA → I/O → CPU → ... (provides better fairness)

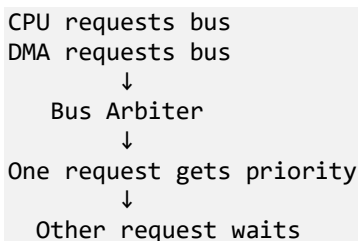
Step 5: Calculate Bandwidth

$$\text{Bandwidth} = \text{Bus Width} \times \text{Clock Frequency}$$

Example: Bus width = 64 bits, Clock frequency = 100 MHz

$$64 \times 100 \text{ MHz} = 6.4 \text{ Gbps} \quad (\approx 800 \text{ MB/s})$$

Step 6: Simulate Transfer Conflict



The waiting time reduces effective throughput.

Result

A hierarchical bus reduces the effect of slow I/O operations on CPU-memory communication. Arbitration resolves conflicts, while greater bus width and frequency increase bandwidth.

Experiment 3: Superscalar and Out-of-Order Processor

Aim

To develop a processor model supporting multiple instruction issue, out-of-order execution, speculative execution, and simultaneous multithreading, and to analyze instruction throughput and resource utilization.

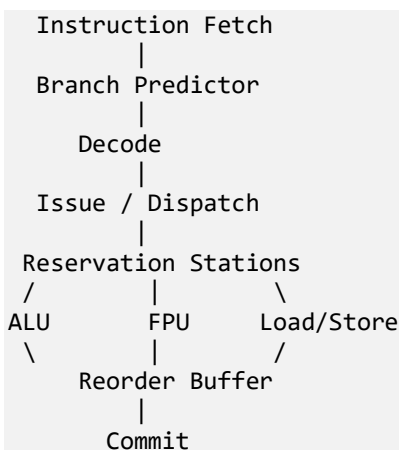
Requirements

- Superscalar processor model
- Reservation Station
- Reorder Buffer
- ALUs
- FPU
- Load/Store Unit
- Branch Predictor
- Multiple hardware threads

Procedure

Step 1: Design the Processor

Use a 4-wide superscalar processor — up to 4 instructions can be issued per cycle, subject to dependencies and available resources.



Step 2: Add Execution Units

ALU 1, ALU 2, FPU, Load/Store Unit — different instructions can execute simultaneously.

Step 3: Implement Out-of-Order Execution

I1: ADD R1,R2,R3

I2: MUL R4,R5,R6

I3: SUB R7,R1,R8

I3 depends on I1, therefore:

I1 → waits/executes

I2 → can execute independently

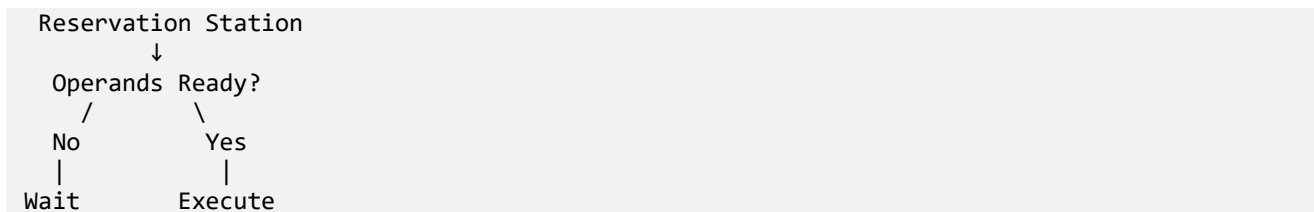
I3 → waits for I1

The processor does not have to stop the entire pipeline.

Step 4: Add Reservation Station

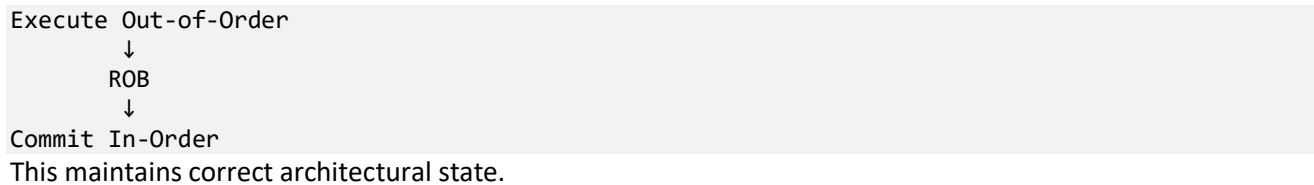
Instructions wait in the reservation station until operands become available.

Instruction
↓

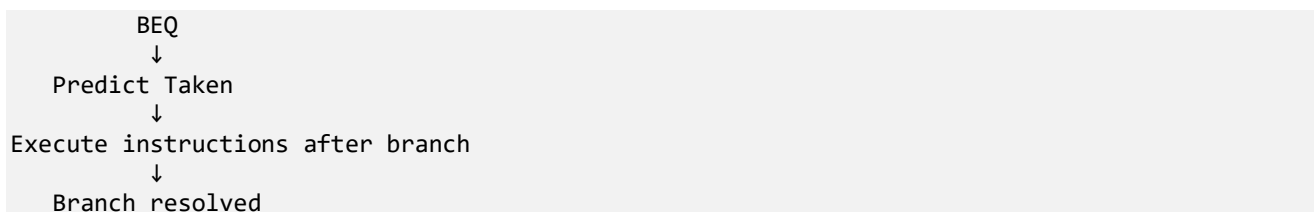


Step 5: Add Reorder Buffer

The ROB ensures results are committed in the original program order.



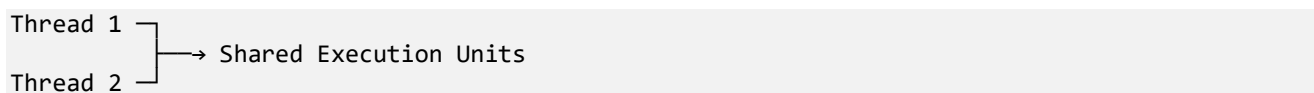
Step 6: Add Speculative Execution



If correct → commit results. If incorrect → flush speculative instructions.

Step 7: Add SMT

Use two hardware threads:



When Thread 1 is waiting for memory, Thread 2 can use available execution units.

Result

The processor improves instruction-level parallelism, throughput, execution-unit utilization, and latency hiding. However, resource contention and branch misprediction can reduce performance.

Experiment 4: Processor Datapath and Control Signals

Aim

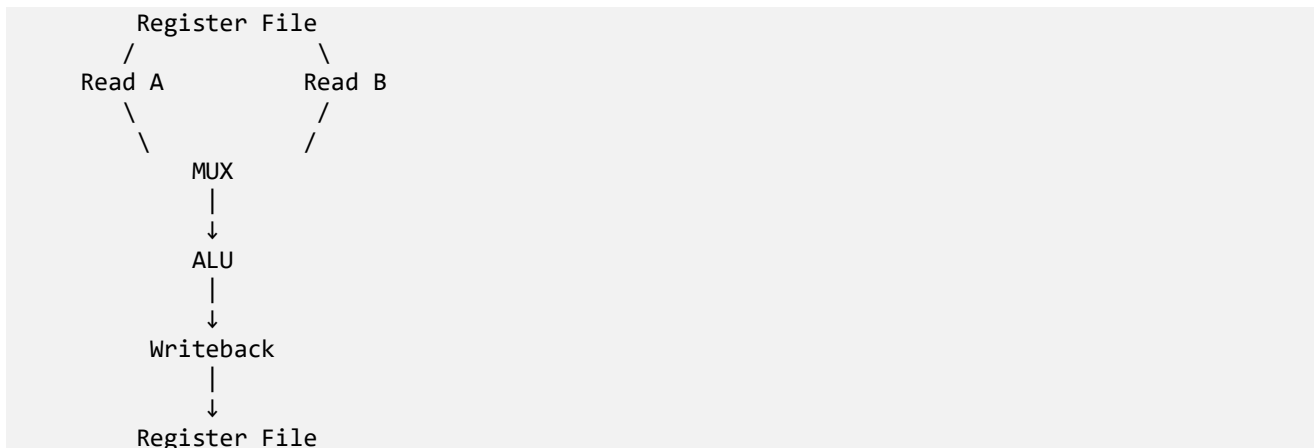
To design and simulate a processor datapath that performs register transfers and arithmetic operations such as increment, decrement, addition, and subtraction.

Requirements

- Register file
- ALU
- Multiplexers
- Control unit
- PC
- Internal buses
- Clock

Procedure

Step 1: Construct Datapath



Step 2: Register-to-Register Transfer

$R3 \leftarrow R1$

Read R1
↓
Select R1
↓
Internal Bus
↓
Select R3
↓
Write R3
Control: RegWrite = 1

Step 3: Addition

$R3 \leftarrow R1 + R2$

R1 → ALU input A
R2 → ALU input B
↓
ADD
↓
R3

Typical control: ALUSrc = 0, ALUOp = ADD, RegWrite = 1, MemToReg = 0

Step 4: Subtraction

$R3 \leftarrow R1 - R2$

R1 → ALU A
R2 → ALU B
↓
SUB
↓
R3

Typical ALU control: SUB → 0110

Step 5: Increment

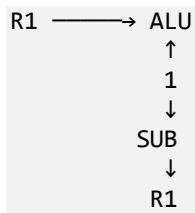
$R1 \leftarrow R1 + 1$

R1 → ALU
↑
1
↓
ADD

↓
R1

Step 6: Decrement

$$R1 \leftarrow R1 - 1$$



Step 7: Control Signals

Operation	RegWrite	ALUSrc	ALU Operation	MemToReg
$R3 \leftarrow R1$	1	—	Transfer	0
$R3 \leftarrow R1 + R2$	1	0	ADD	0
$R3 \leftarrow R1 - R2$	1	0	SUB	0
$R1 \leftarrow R1 + 1$	1	1	ADD	0
$R1 \leftarrow R1 - 1$	1	1	SUB	0

Result

The datapath successfully performs register transfers and arithmetic operations. Control signals determine the data path and ALU operation for each instruction.

Experiment 5: Pipelined Processor Performance

Aim

To simulate a pipelined processor executing arithmetic, memory, and branch instructions, and to calculate execution time, throughput, and speedup compared with sequential execution.

Requirements

- 5-stage pipeline
- Instruction sequence
- Clock cycle
- Pipeline timing table
- Calculator

Procedure

Step 1: Define Pipeline

IF → ID → EX → MEM → WB

$$k = 5$$

Step 2: Define Instructions

I1: ADD R1,R2,R3
I2: SUB R4,R1,R5
I3: LW R6,0(R7)
I4: ADD R8,R6,R9

I5: BEQ R8,R10,LABEL

I6: SW R8,0(R11)

Arithmetic → ADD/SUB, Memory → LW/SW, Branch → BEQ

Step 3: Create Pipeline Table

For an ideal pipeline:

Instr.	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
I1 ADD	IF	ID	EX	MEM	WB					
I2 SUB		IF	ID	EX	MEM	WB				
I3 LW			IF	ID	EX	MEM	WB			
I4 ADD				IF	ID	EX	MEM	WB		
I5 BEQ					IF	ID	EX	MEM	WB	
I6 SW						IF	ID	EX	MEM	WB

For 6 instructions and 5 stages:

$$T_{\text{pipe}} = k + N - 1 = 5 + 6 - 1 = 10 \text{ cycles}$$

Step 4: Calculate Sequential Time

If every instruction requires 5 cycles:

$$T_{\text{seq}} = N \times k = 6 \times 5 = 30 \text{ cycles}$$

Step 5: Calculate Speedup

$$\text{Speedup} = T_{\text{seq}} / T_{\text{pipe}} = 30 / 10 = 3$$

The idealized pipeline provides a 3× speedup for this six-instruction sequence under these assumptions.

Step 6: Calculate Throughput

If the clock cycle $t_c = 1$ ns, then $T_{\text{pipe}} = 10$ ns, so:

$$\text{Throughput} = N / (T_{\text{pipe}} \times t_c) = 6 / 10 \text{ ns} = 0.6 \text{ instructions/ns}$$

≈ 600 million instructions per second.

Step 7: Include Hazards

I1 → I2 Data dependency → Forwarding
I3 → I4 Load-use dependency → 1-cycle stall
I5 Branch → Branch prediction

So the actual execution time becomes:

$$T_{\text{actual}} = T_{\text{ideal}} + \text{stall cycles} + \text{flush cycles}$$

For example, with 1 stall cycle and 2 branch-flush cycles:

$$T_{\text{actual}} = 10 + 1 + 2 = 13 \text{ cycles}$$

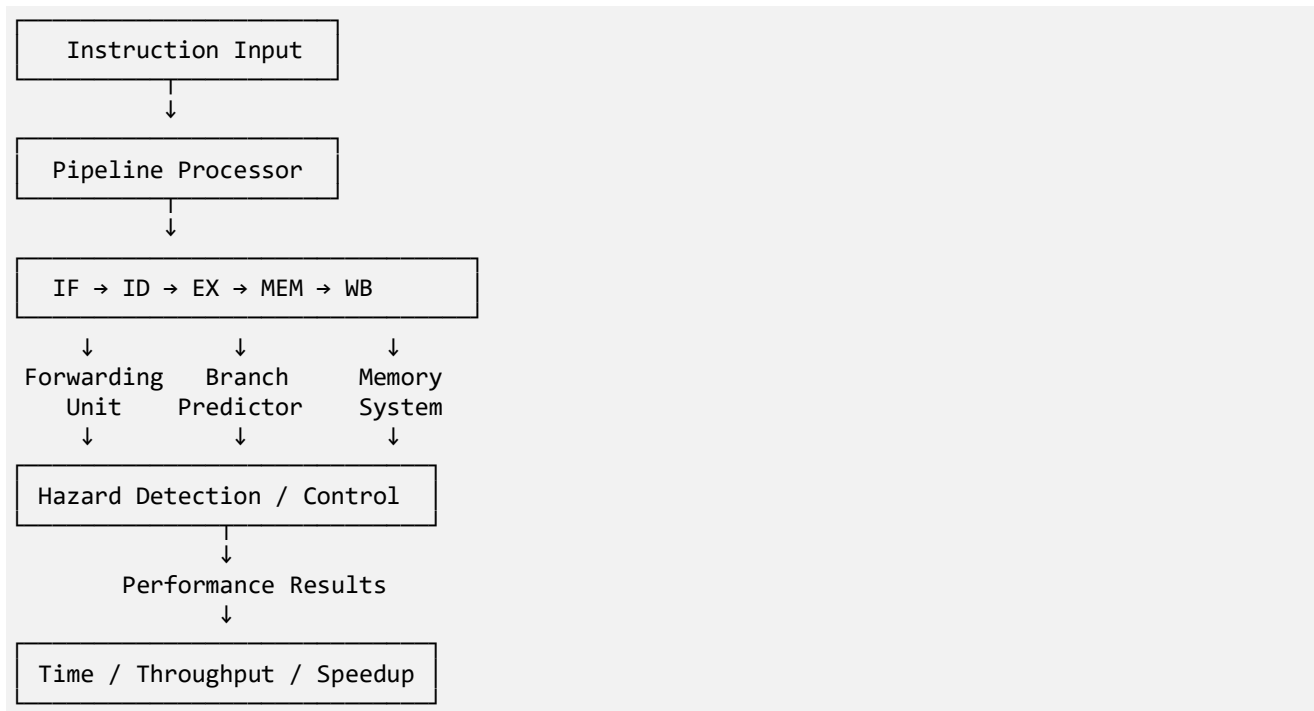
$$\text{Speedup} = 30 / 13 \approx 2.31$$

This demonstrates that hazards reduce the actual speedup from the ideal value.

Result

Pipelined execution showed higher throughput and lower execution time than sequential execution, although hazards and branch penalties reduced the ideal speedup from 3× to approximately 2.31×.

Overall Experimental Setup



Overall Result

The processor models were successfully constructed and analyzed. Pipeline hazards were minimized using operand forwarding, pipeline stalling, branch prediction, and resource duplication. A hierarchical bus structure was used to study bandwidth, priority, and transfer conflicts. A superscalar processor model demonstrated multiple issue, out-of-order execution, speculative execution, and SMT. The datapath successfully performed register transfers and arithmetic operations. Finally, pipelined execution showed higher throughput and lower execution time than sequential execution, although hazards and branch penalties reduced the ideal speedup.